

YAZILIM MÜHENDİSLİĞİ

HAZIRLAYAN VE SUNAN

Doç. Dr. Asım Sinan YÜKSEL



asimyuksel@sdu.edu.tr

4. Ders



SÜLEYMAN
DEMİREL
UNİVERSİTESİ



BİLGİSAYAR
MÜHENDİSLİĞİ

CI/CD Kavramı

- **CI/CD:** Continuous Integration/Continuous Deploy/Delivery: Sürekli Entegrasyon/Sürekli Teslim/Dağıtım
- Günümüz dünyasında sürekli değişen teknoloji karşısında şirketler ayakta kalma ve rekabet etme mücadelesi vermektedirler.
- Sürdürülebilir yönetim ve hızlı yazılım teslimi arasındaki boşluk şirketler tarafından doldurulmalıdır.
- CI/CD bu noktada devreye girerek hem istikrarlı ve güvenli bir sistem sağlarken hem de hızlı bir şekilde yazılım değişikliklerini yapabilmeyi sağlar.
- **CI: Sürekli Entegrasyon:**
 - Geliştiriciler sık sık kodlarındaki değişiklikleri birleştirirler (merge) ve merkezi kod deposuna gönderirler.
 - Daha sonra otomatik testler üretilir ve çalıştırılır.
 - CI genellikle yazılımın dağıtım sürecindeki aşamasında gerçekleşen entegrasyon işlemini ifade eder.
 - Ana hedef hataların hızlı bir şekilde bulunması ve çözülmesi, yazılım kalitesinin artırılması, yeni yazılım güncellemelerinin dağıtılması için gerekli süreyi kısaltmak.
 - Küçük kod değişikliklerine ve küçük kod işlemelerine (commit) odaklanır.

CI/CD Kavramı

- **CD: Sürekli Dağıtım/Teslim:**

- Yazılım değişiklikler sonrası üretim ortamına çıkmak için hazırlanır, derlenir, test edilir.
- Tamamen otomatik ya da önemli noktaların el ile yapıldığı bir süreç halinde işletilebilir.
- Değişiklikler/revizyonlar üretim ortamına başka bir yazılımcının onayı olmadan otomatik bir şekilde üretim ortamına aktarılabilir.
- Tüm yazılım dağıtım süreci otomatik hale gelir.
- Üretim yaşam döngüsünde müşteriden daha fazla geri dönüş alınmasını sağlar.
- Sürekli teslimat dendiğinde her değişikliğin işlendiği (commit) ve otomatik testi geçtikten hemen sonra üretim ortamına aktarıldığı bir süreç anlaşılmamalı.
- Sürekli teslimat her değişikliğin üretim ortamına çıkmak için hazır hale geldiğinden emin olunmasını sağlar.
- Sürekli teslimatta canlıya geçme kararı ticari bir karardır, teknik bir karar değildir.
- Teknik kararlar her kod işleme aşamasında gerçekleşir.
- Dağıtımda teknik takım çalışmayı durdurmaz, diğer aşamalara geçmek için beklemez, proje planına, dokümantasyona ihtiyaç duyulmaz.
- Dağıtım tekrarlanan bir süreç haline gelir ve yazılım doğruluğu test ortamlarında çok defa test edilir.

CI/CD Kavramı



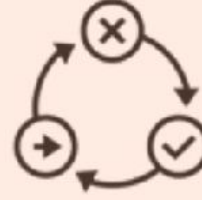
Sürekli Herşey/Continuous Everything



Sürekli Entegrasyon (CI)



Sürekli Dağıtım (CD)



Sürekli Test



Sürekli Teslim (CD)

Otomatik ve Artırımsal

- Geliştirme sürecinin en erken sürecinde hataların tespiti
- Kısa derleme süresi

- Küçük bir kod değişikliğinde esnek test imkanı
- Dağıtım öncesi testi teşvik eder

- Her kod değişikliğinde gerekli testler çalışır
- Kaliteli kod
- Hızlı hata giderme

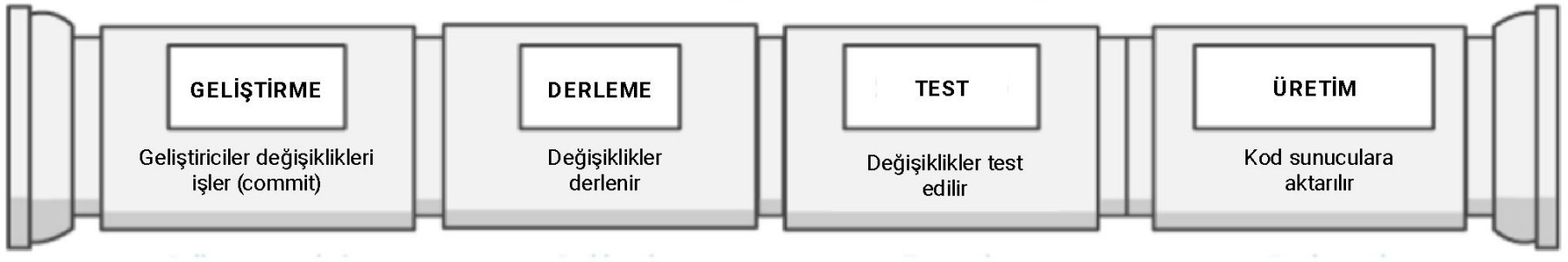
- Düşük riskli ve acısız bir şekilde uygulama teslimi
- Daha kolay yönetim ve genişleme

CI/CD Kavramı

- **Faydaları:**

- Yazılım ortaya çıkarma süreci otomatikleşir.
- Geliştirici üretkenliğini artırır.
- Kod kalitesini artırır.
- Güncellemeler daha hızlı teslim edilir.
- CI/CD süreci bir boru hattı (pipeline)/olarak düşünülebilir.
- Pipeline'daki her bir aşama mantıksal bir birim olarak düşünülebilir.
- Kod hat boyunca ilerledikçe kod kalitesi her aşamada artar.
- Erken aşamalarda karşılaşılan bir hata sonraki aşamalara geçilmesini engeller.
- Test sonuçları takımlara anında iletilir. Testler geçilmezse yeni derlemeler, dağıtımlar durur.
- Pipeline'daki aşamalar sadece önerilerden oluşur. İş gereksinimlerine göre değişebilir.
- CI/CD Pipeline'ın varlığı şirketin olgunlaşmasına büyük katkı sağlayacaktır. Ancak şirketler tam olgunlaşmış bir pipeline yerine küçük adımlarla başlayarak süreci işletmelidirler.

CI/CD Kavramı



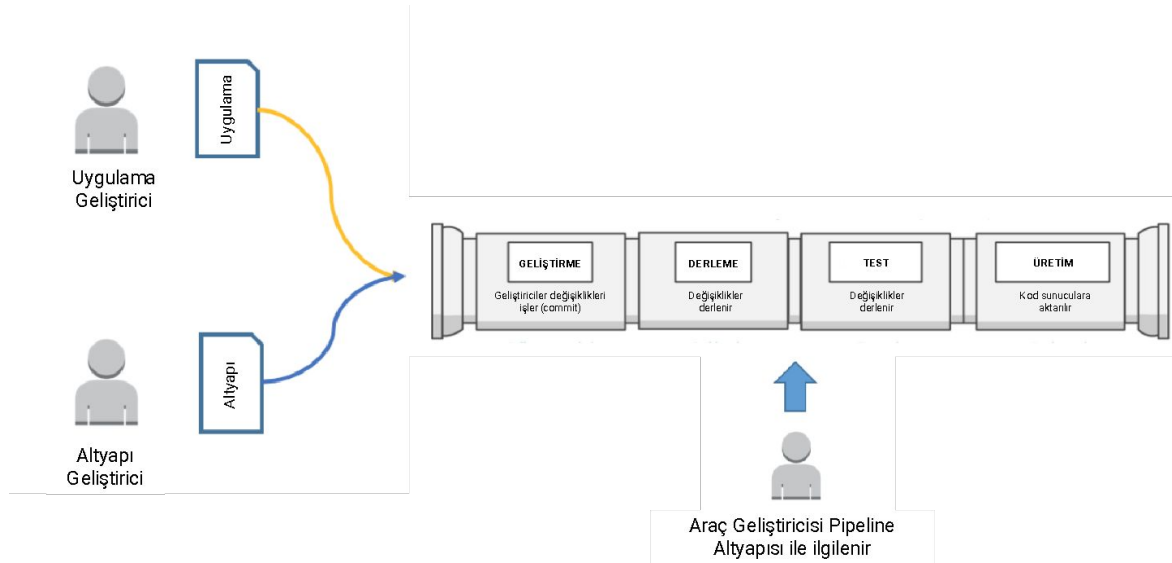
CI/CD Pipeline

CI/CD Kavramı

- CI/CD bir yolculuktur. Bu yolculukta hedef yazılımı olgunlaştırmaktır.
- Tüm yazılımcılar sık sık kod değişikliklerini merkezi kod deposuna (repository) işlemelidirler. Tüm değişiklikler uygulamanın dağıtım dalında (branch) birleştirilmelidir (merge).
- Kodu erken birleştiren geliştirici yol boyunca entegrasyon problemleriyle daha az karşılaşacaktır.
- Geliştiriciler birim testlerini olabildiğince erken oluşturmaları ve bu işlemi kodlarını merkezi kod deposuna göndermeden önce yapmalarıdır.
- Erken aşamalarda yakalanan hataların çözülmesi daha kolay, maliyeti daha azdır.
- Kod depodaki bir dala itildiğinde (push) o dalı izleyen akış yöneticisi derleme aracına derleme işleminin başlaması ve kontrollü bir ortamda testlerin çalıştırılması için emir verir.
- Sonuçta derlenmiş bir dosya (binary) oluşur.
- Bir sonraki aşama teslim aşamasıdır. Kod üretim ortamının bir kopyası olan test ortamına aktarılır.
- Son aşama ise dağıtım aşamasıdır. Yazılım üretim ortamına aktarılır.
- **Dip Not:** HP LaserJet Firmware takımına göre çevik yaklaşımla birlikte CI/CD sürecini benimsemek program başına %78 oranında kazanç sağlayabilmektedir.

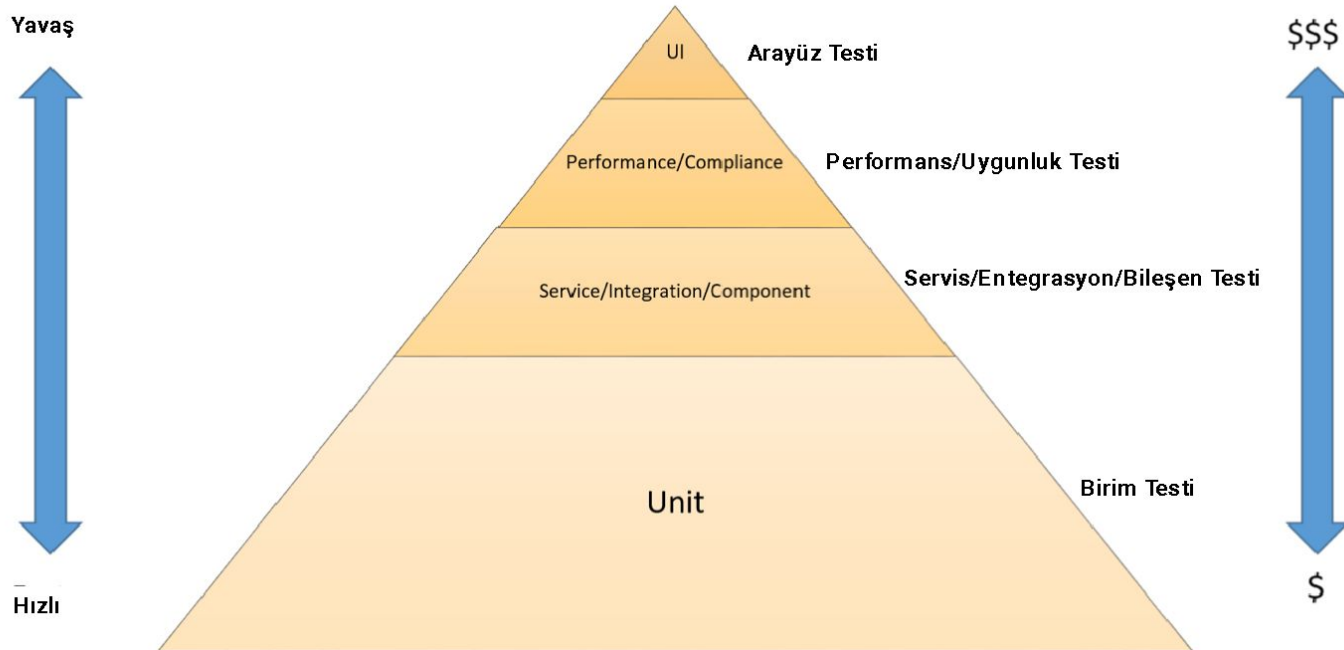
CI/CD Kavramı

- CI/CD sürecinde takımların rolü büyüktür. Bu süreçte büyük tecrübesi olan Amazon firması CI/CD ortamının oluşturulması için takımların 3 ayrı geliştirici takıma bölünmesini tavsiye etmektedir. Takımlar maksimum 10-12 kişiden oluşmalıdır.
- Bunlar:
 - **Uygulama Takımı (Application Team):** Uygulamayı geliştirir. İş Listesi (Backlog), Hikayeler (Stories), Birim Testlerinden (Unit Tests) sorumludur.
 - **Altyapı Takımı (Infrastructure Team):** Uygulamanın çalışması için gerekli altyapıyı oluşturacak, altyapı ayarlarını yapacak kodu yazar.
 - **Araçlar Takımı (Tools Team):** CI/CD pipeline'ı yönetir. Pipeline'ı oluşturan araçlardan ve altyapıdan sorumludurlar.



CI/CD Kavramı

- Bu 3 takım yazılım geliştirme sürecinde pipeline'ın çeşitli aşamalarında test işlemlerini sürece dahil etmelidirler.
- Test olabildiğince erken başlamalıdır.
- Aşağıda test aşamasının hız ve maliyet açısından önemini gösteren ve Mike Cohn tarafından "Succeeding with Agile" isimli kitapta yer alan test piramidi yer almaktadır.



CI/CD Kavramı

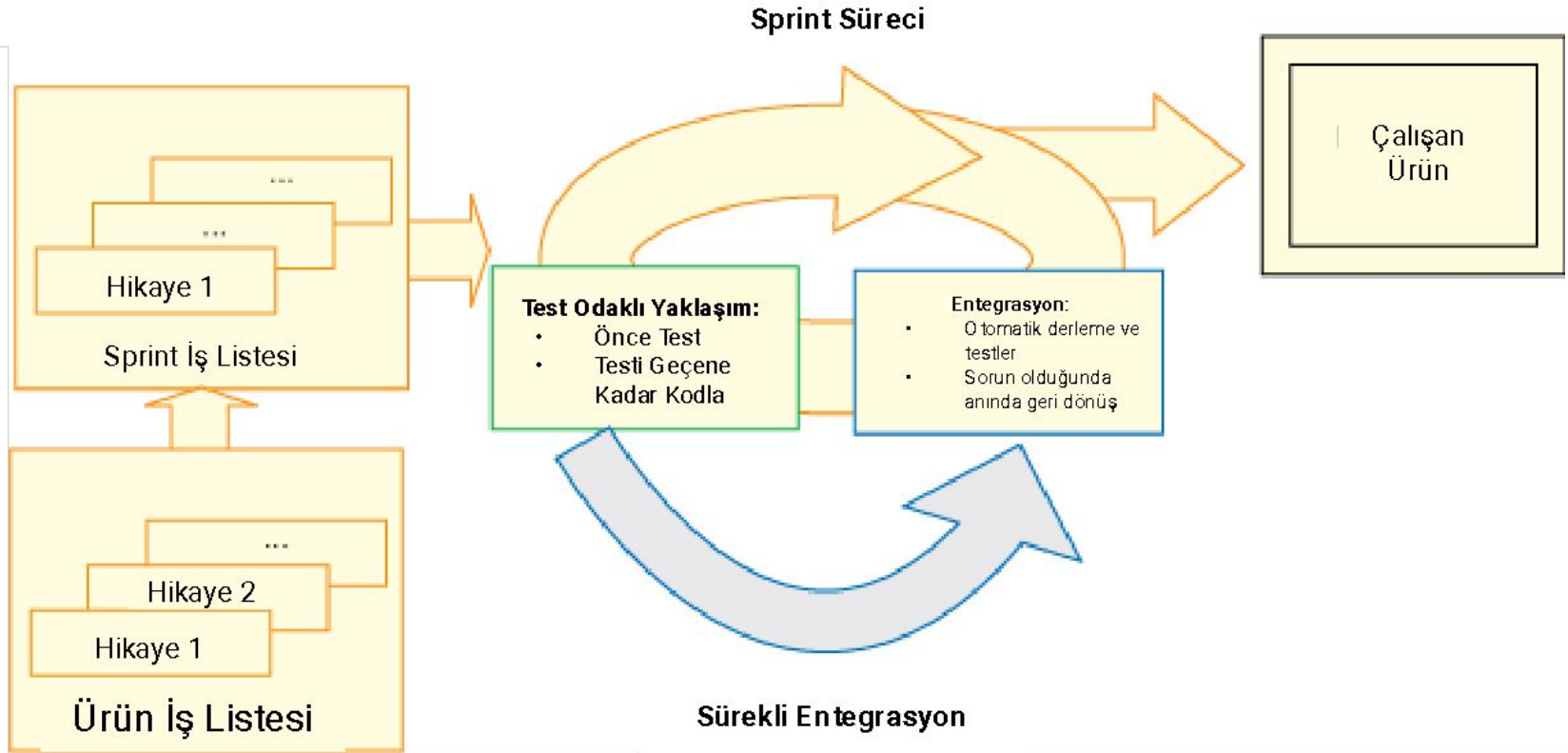
- Bu diyagrama göre birim testi piramidin en altında yer alır. En hızlı çalışırlar ve en düşük maliyetli olanlardır.
- Dolayısıyla birim testleri test stratejinizin büyük bir kısmını (%70) oluşturmmalıdır.
- Servis entegrasyon ve bileşen testleri birim testinin üstündedir.
- Bu testler detaylı bir ortama ihtiyaç duyarlar. Altyapı gereksinimleri maliyetlidir. Çalışmaları yavaştır.
- Arayüz ve kullanıcı kabul testleri piramidin en üstünde yer alırlar. Üretim kalitesinde bir ortam gerektirirler.
- **CI/CD aşamaları:**
 - **Geliştirme:** Kodunuzun ve konfigürasyon ayarlarınızın yapılacağı, depolanacağı bir depo (Github gibi) ayarlanır.
 - **Derleme:** En uygun derleme araçları bulunarak derleme işlemi gerçekleştirilir. Uygulama bağımlılıkları net bir şekilde tanımlanır. Java için Maven, Gradle; C/C++ için Make; Javascript için Grunt; Ruby için Rake.
 - **Test:** Üretim ortamının birebir kopyası olan bir ortam oluşturulur. Entegrasyon, bileşen, sistem, performans, uygunluk, kullanıcı kabul testleri gerçekleştirilir.
 - **Üretim:** Test aşamasında yapılanlar üretim ortamında tekrarlanır.

CI/CD Kavramı

CI/CD Öncesi	CI/CD Sonrası
Yıllık dağıtım	Sık dağıtım
Şelale modeli	Çevik model
İzole edilmiş kodlama	Ortak bir depoya sık sık kod gönderme
Kod birleştirme karmaşası	Sıkıntısız kod birleştirme
El ile test	Otomatik test
Üretim öncesi karmaşa	Kod her zaman üretime hazır
El ile dağıtım	Otomatik dağıtım
Geç geri dönüş süresi	Hızlı geri dönüş döngüsü

CI/CD Kavramı

Test Odaklı Yaklaşım (TDD), CI/CD ve Scrum İş Birliği



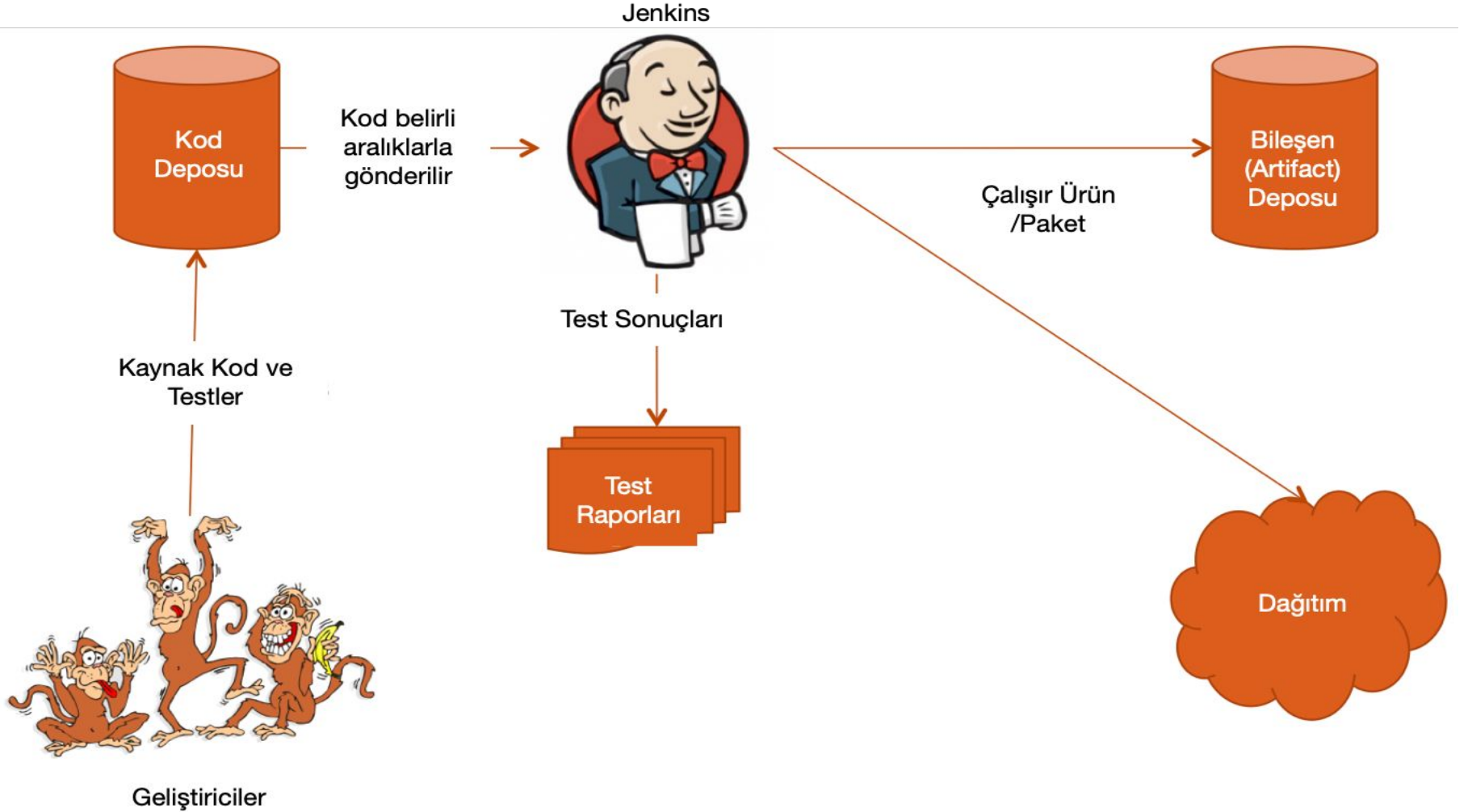
CI/CD Araçları-Jenkins

- 2005 yılında Sun Microsystems tarafından geliştirilen Hudson isimli CI aracından türemiştir.
- Oracle 2009 yılında Sun Microsystems'i satın aldıktan sonra projenin yönetimi de Oracle'a geçmiştir.
- 2011 yılında Hudson temel alınarak Jenkins geliştirilmeye başlanmıştır.
- Java tabanlı Sürekli Geliştirme Sistemidir (Continuous Build System).
- Tomcat ya da GlassFish gibi servlet kapsayıcısında çalışır.
- Son derece yapılandırılabilir bir sistemdir. Geliştiricilerin eklenti geliştirmesine izin verir.
- MIT lisansı altında kullanıma sunulmuştur.
- Büyük bir destek topluluğu vardır.
- Test raporları oluşturmanızı sağlar.
- Çok farklı versiyon kontrol sistemleriyle entegre olabilir.
- Farklı çıktı/yazılım bileşeni depolarına (artifect repository) gönderim yapabilir.
- Doğrudan test ve üretim ortamına dağıtım yapabilir.
- Paydaşlara geliştirme süreci hakkında bildirim gönderebilir.

CI/CD Araçları-Jenkins

- Jenkins'te proje oluşturduğunuzda aşağıdaki özelliklerin hepsi içinde gelmektedir.
 - Versiyon kontrol sistemleriyle ilişkilendirme
 - Geliştirmeyi/Yapılandırmayı/Derlemeyi tetikleme
 - Kabul skriptlerinin, bash skriptlerinin, Ant veya Maven hedeflerinin çalıştırma
 - Yazılım bileşeni/çıktı (Artifact) arşivleme
 - Birim testlerini ve Javadoc dokümanlarını yayınlama
 - E-Posta bildirimleri
- Projeyi başarılı bir şekilde oluşturduğunuzda gelecekteki tüm derleme işlemleri otomatik gerçekleşir.
- Raporlama özelliği vardır. Tüm yapılandırma durumları (Son başarılı derleme, son hatalar gibi) hakkında bilgi verebilir.
- Ne kadar birim testi yaptığınızı, test sonuçlarını gösterebilir.
- 2530'dan fazla firma Jenkins'i kullanmaktadır.
- Jenkins Kullanan Büyük Firmalar: Netflix, Facebook, Alcatel, Atmel, Dell, Deutsche Telekom, Ebay, Github, LinkedIn, Motorola, Nokia, Sony, SpaceX, İngiliz Hükümeti, Yahoo ve daha fazlası...

CI/CD Araçları-Jenkins



CI/CD Jenkins NodeJS Örnek

- Docker arkaplanda çalışırken Jenkins son sürüm docker imajını aşağıdaki komutla yükleyin:

```
docker run -d \  
  --name jenkins \  
  --restart unless-stopped \  
  -p 8080:8080 \  
  -p 50000:50000 \  
  -v jenkins_home:/var/jenkins_home \  
  -v /var/run/docker.sock:/var/run/docker.sock \  
  jenkins/jenkins:its
```


CI/CD Jenkins NodeJS Örnek

- Jenkins container icine Docker CLI kur:

GPG anahtarini indir

```
docker exec -u root jenkins bash -c "install -m 0755 -d  
/etc/apt/keyrings && curl -fsSL  
https://download.docker.com/linux/debian/gpg -o  
/etc/apt/keyrings/docker.asc && chmod a+r  
/etc/apt/keyrings/docker.asc"
```

Docker reposunu ekle

```
docker exec -u root jenkins bash -c 'ARCH=$(dpkg  
--print-architecture) && CODENAME=$(. /etc/os-release && echo  
$VERSION_CODENAME) && echo "deb [arch=$ARCH  
signed-by=/etc/apt/keyrings/docker.asc]  
https://download.docker.com/linux/debian $CODENAME stable" >  
/etc/apt/sources.list.d/docker.list'
```

Docker CLI ve Compose kur

```
docker exec -u root jenkins bash -c "apt-get update && apt-get  
install -y docker-ce-cli docker-compose-plugin"
```

CI/CD Jenkins NodeJS Örnek

- Docker socket yetkisini ayarla:

```
docker exec -u root jenkins chmod 666 /var/run/docker.sock
```

- Test et:

```
docker exec jenkins docker --version
```

```
docker exec jenkins docker compose version
```

CI/CD Jenkins NodeJS Örnek

- Jenkinsfile dosyası oluştur ve repoya push et.

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/asimsinan/mekanbul.git'
      }
    }

    stage('Build and Deploy') {
      steps {
        sh 'docker compose down'
        sh 'docker compose up -d --build'
      }
    }

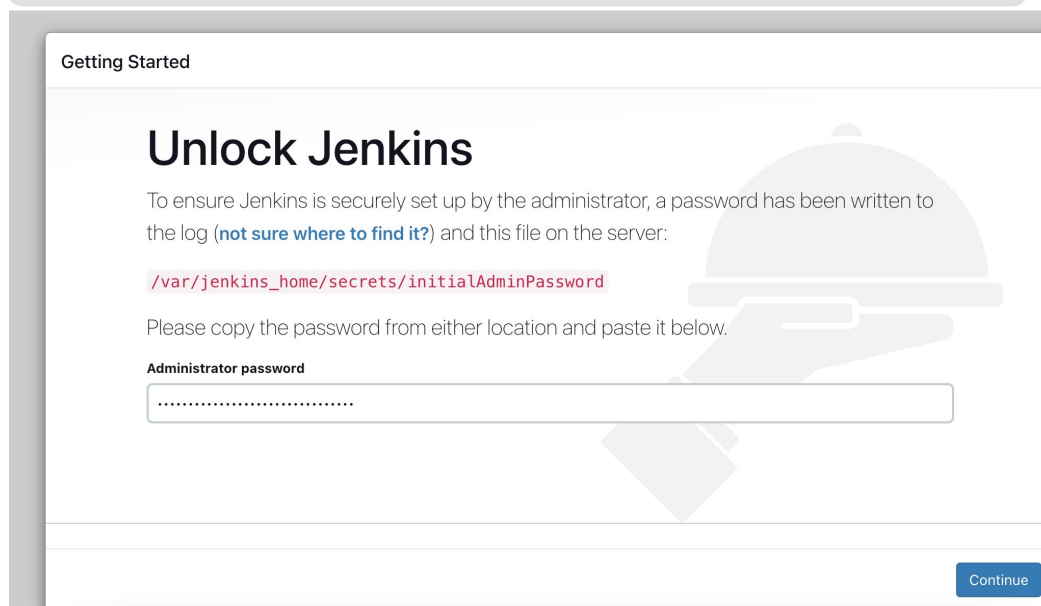
    stage('Health Check') {
      steps {
        script {
          sleep 10
          sh 'curl -f http://localhost:3000 || echo "Backend henuz hazır değil"'
        }
      }
    }
  }

  post {
    success {
      echo 'Deploy basarili: mekanbul calisiyor.'
    }
    failure {
      echo 'Deploy basarisiz: loglari kontrol et.'
    }
  }
}
```

CI/CD Jenkins NodeJS Örnek

- Loglarda yer alan admin şifresini localhost:8080 adresini açarak girin.

```
2023-04-11 12:52:52 WARNING: Please consider reporting this to the maintainers of org.codehaus.groovy.vmpugin.v7..
2023-04-11 12:52:52 WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access ope
2023-04-11 12:52:52 WARNING: All illegal access operations will be denied in a future release
2023-04-11 12:52:53 2023-04-11 09:52:53.346+0000 [id=28] INFO jenkins.install.SetupWizard#init:
2023-04-11 12:52:53
2023-04-11 12:52:53 *****
2023-04-11 12:52:53 *****
2023-04-11 12:52:53 *****
2023-04-11 12:52:53
2023-04-11 12:52:53 Jenkins initial setup is required. An admin user has been created and a password generated.
2023-04-11 12:52:53 Please use the following password to proceed to installation:
2023-04-11 12:52:53
2023-04-11 12:52:53 49c4f3ea3c204632ad2c744442dbf42b
2023-04-11 12:52:53
2023-04-11 12:52:53 This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
2023-04-11 12:52:53
2023-04-11 12:52:53 *****
localhost:8080/login?from=%2F
```



Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

`/var/jenkins_home/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

Continue

CI/CD Jenkins NodeJS Örnek

- Pluginlerin yüklenmesini bekleyin

Getting Started

Getting Started

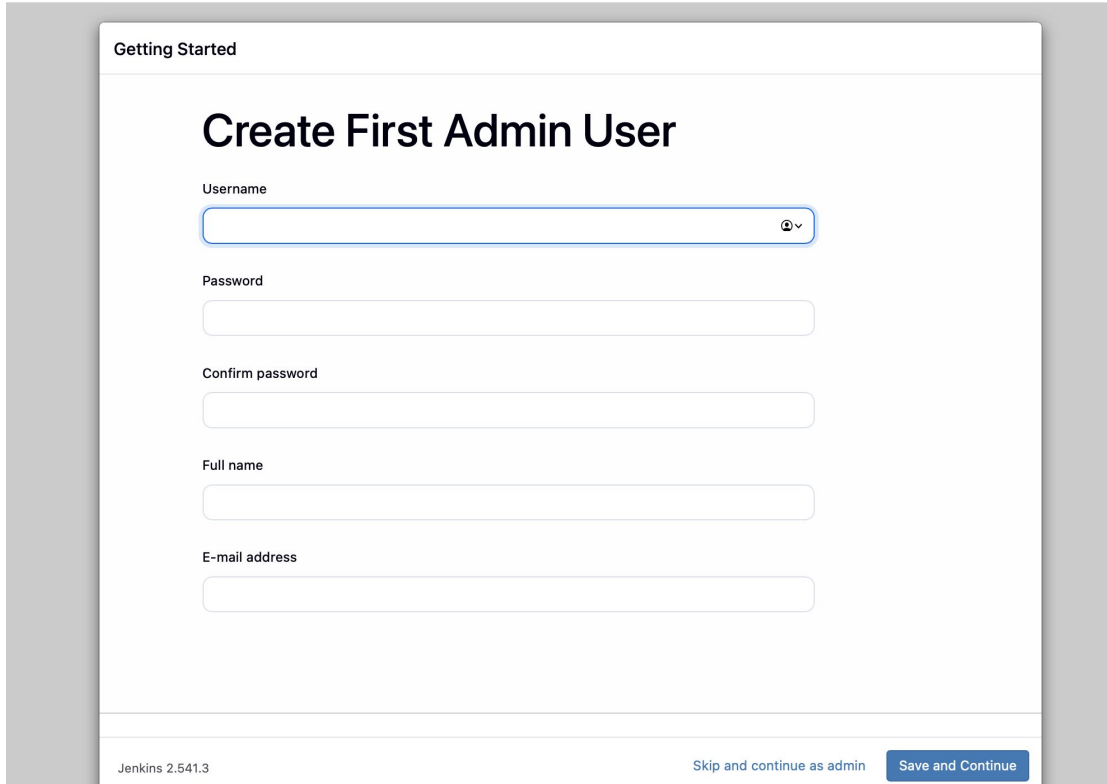
✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	✓ Credentials Binding	<pre>** Matrix Project ** Resource Disposer Workspace Cleanup Ant ** OkHttp ** Apache HttpComponents Client 4.x API ** Pipeline: Job Gradle ** Pipeline: Milestone Step ** Durable Task ** Pipeline: Nodes and Processes ** Pipeline: Build Step ** Pipeline: SCM Step ** Pipeline: Groovy ** Pipeline: Groovy Libraries ** Joda Time API ** Pipeline: Model API ** Pipeline: Stage Step ** Pipeline: Declarative Extension Points API ** Jakarta Mail API ** Instance Identity Mailer ** Branch API ** Pipeline: Multibranch ** Pipeline: Stage Tags Metadata ** Pipeline: Input Step ** - required dependency</pre>
✓ Timestamper	✓ Workspace Cleanup	✓ Ant	✓ Gradle	
🔄 Pipeline	🔄 GitHub Branch Source	🔄 Pipeline: GitHub Groovy Libraries	🔄 Pipeline Graph View	
🔄 Git	🔄 SSH Build Agents	🔄 Matrix Authorization Strategy	🔄 LDAP	
🔄 Email Extension	✓ Mailer	🔄 Dark Theme		

Jenkins 2.541.3

Safari

CI/CD Jenkins NodeJS Örnek

- Aşağıdaki «Skip and continue as admin» tuşa basarak devam edin.



The image shows the Jenkins 'Getting Started' screen. At the top, it says 'Getting Started'. Below that, the main heading is 'Create First Admin User'. There are five input fields: 'Username' (with a dropdown arrow), 'Password', 'Confirm password', 'Full name', and 'E-mail address'. At the bottom left, it says 'Jenkins 2.541.3'. At the bottom right, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'.

CI/CD Jenkins Pipeline Örnek



Jenkins / All ▾ / New Item

New Item

Enter an item name

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.



Organization Folder

Creates a set of multibranch project subfolders by scanning for repositories.

OK

CI/CD Jenkins Pipeline Örnek



Jenkins

/ mekanbul



/ Configure

Configure



General



Triggers



Pipeline



Advanced

General

Enabled



Description

Plain text [Preview](#)

☐ Discard old builds ?

☐ Do not allow concurrent builds

☐ Do not allow the pipeline to resume if the controller restarts

☒ GitHub project

Project url ?

<https://github.com/asimsinan/mekanbul/>

Advanced



☐ Pipeline speed/durability override ?

☐ Preserve stashes from completed builds ?

☐ This project is parameterized ?

CI/CD Jenkins Pipeline Örnek



Jenkins / mekanbul / Configure

Configure



General



Triggers



Pipeline



Advanced

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?
- ☐ Trigger builds remotely (e.g., from scripts) ?

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/asimsinan/mekanbul/

CI/CD Jenkins Pipeline Örnek

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

+ Add

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

SaveApply

CI/CD Jenkins Pipeline Örnek

Quick search... %K

Account home

Recents >

Analytics & logs >

Domains >

Build

Compute >

AI >

Storage & databases >

Media >

Protect & Connect

Application security >

Zero Trust

Networking >

Insights >

Tunnels New

Create a Tunnel

Start by creating a Tunnel to connect HTTP web servers, SSH servers, remote desktops, and other protocols securely to Cloudflare.

Tunnel name ⓘ

a

Setup Environment

Choose your operating system to get installation instructions.

Operating System *

Windows

macOS

Debian

Red Hat

Docker

Install and Run

Run the following commands to install and connect your Tunnel.

⚠ Security Notice

This token grants access to your account. Keep it secure and don't share it.

Install cloudflared

```
$ brew install cloudflared && sudo cloudflared service install e
```

CI/CD Jenkins Pipeline Örnek

```
asimsinanyuksel@Asms-MacBook-Pro Jenkins % cloudflared tunnel --url http://localhost:8080
2026-04-07T09:39:01Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloud
flare account, is a quick way to experiment and try it out. However, be aware that these a
ccount-less Tunnels have no uptime guarantee, are subject to the Cloudflare Online Service
s Terms of Use (https://www.cloudflare.com/website-terms/), and Cloudflare reserves the ri
ght to investigate your use of Tunnels for violations of such terms. If you intend to use
Tunnels in production you should use a pre-created named tunnel by following: https://deve
lopers.cloudflare.com/cloudflare-one/connections/connect-apps
2026-04-07T09:39:01Z INF Requesting new quick Tunnel on trycloudflare.com...
2026-04-07T09:39:06Z INF +-----+
+-----+
2026-04-07T09:39:06Z INF | Your quick Tunnel has been created! Visit it at (it may take s
ome time to be reachable): |
2026-04-07T09:39:06Z INF | https://helena-substitute-yeah-cds.trycloudflare.com
|
2026-04-07T09:39:06Z INF +-----+
+-----+
```

CI/CD Jenkins Pipeline Örnek

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models Preview

Webhooks

Copilot

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

Secrets and variables

Webhooks / Manage webhook

Settings Recent Deliveries

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

https://helena-substitute-yeah-cds.trycloudflare.com/github-webhook/

Content type *

application/x-www-form-urlencoded

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

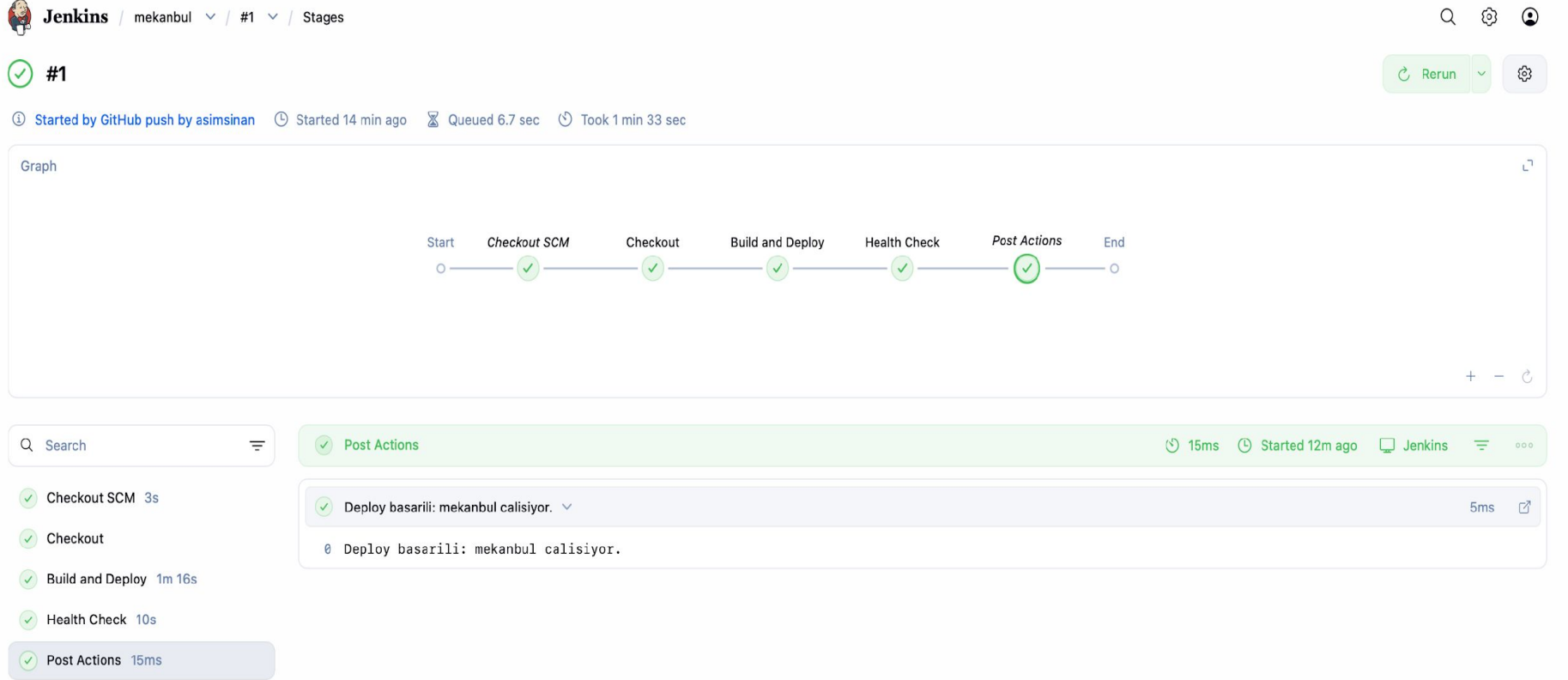
☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☐ Just the push event.

☒ Send me everything.

CI/CD Jenkins Pipeline Örnek



CI/CD ve Docker Kısımları Final İçin Beklenenler

- Projenin CI/CD kısmı tüm bu aşamaların çalıştığını kanıtlayacak şekilde
- github Sunum (7. madde) kısmına bir youtube video linki olarak eklenmeli.
- Tüm aşamalar için Jenkinsfile dosyası oluşturmalı ve repoda bulunmalı.
- Bu kısım grupça yapılacak ve herkes aynı puanı alacak.
- Front-end ve REST API docker üzerinden yayınlanmalı.
- Lokalde çalışması yeterli.
- REST API ve frontend'in Docker üzerinde çalıştığı videoda gösterilmeli.
- CI/CD ve Docker birlikte değerlendirilecek ve toplam 15 puan değerinde. Herhangi biri eksikse puan verilmeyecek.

CI/CD Kaynaklar

- <https://github.com/ligurio/awesome-ci>
- <https://github.com/ciandcd/awesome-ciandcd>
- <https://github.com/asimsinan/mekanbul>